

System-on-Chip

UCSD ECE 260C Spring 2026

Davit Markarian

dtmar@ucsd.edu

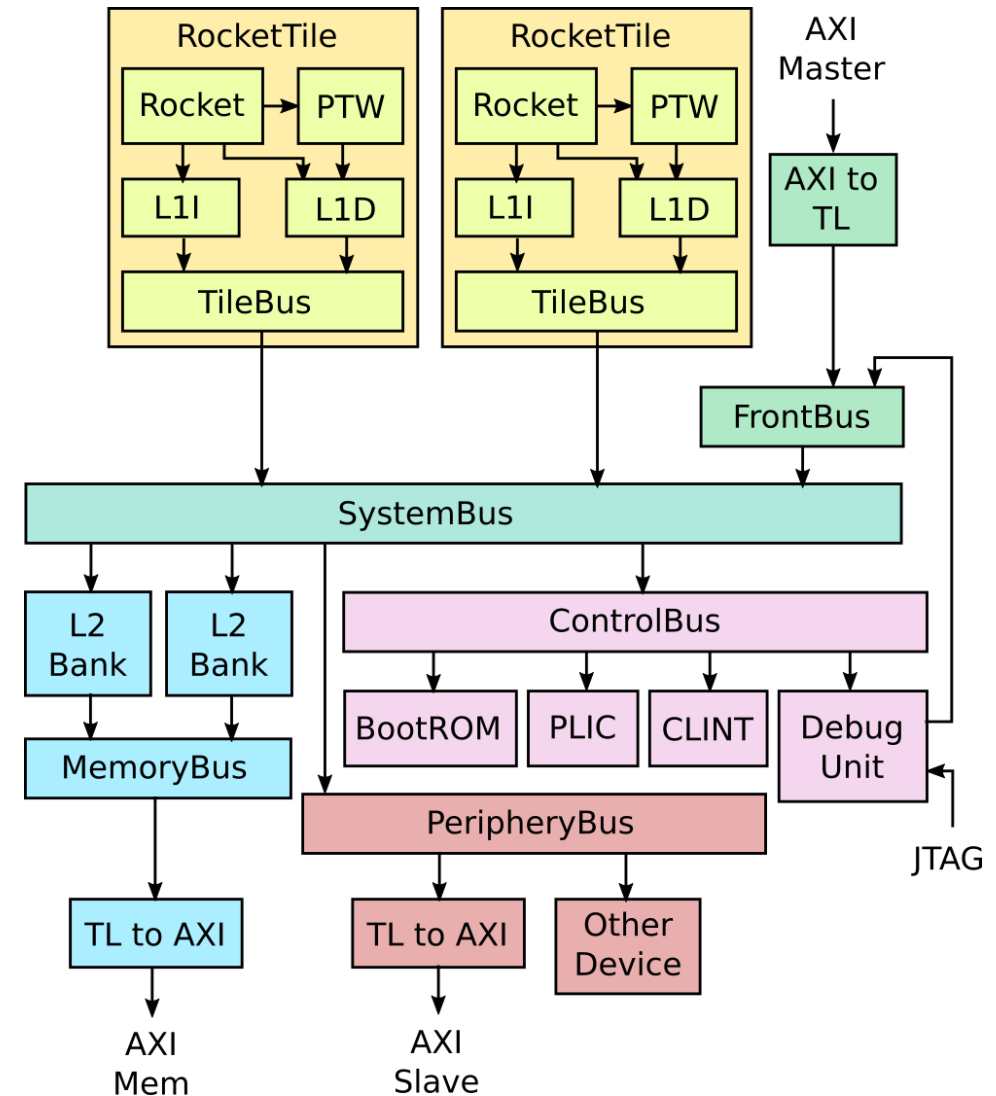
Agenda

- Architectural Decisions
 - Busses, Programming Model, IP Generators, I/O by protocol
- Physical Implementation
 - SoC Floorplanning
 - Design strategies
 - MIIMs and partitioning
 - Yield Recovery
 - Dataflow
 - Core Floorplanning
 - Technology Mapping
 - Clock & Power Planning
 - Logic Implementation
 - Miniproject 2

Architectural Decisions

Decisions

- SoCs present a wide architectural space...
 - Compute cores
 - Memory hierarchy
 - Accelerators
 - Busses/DMA units
 - I/Os
- ... And these drive a wide range of implementation possibilities
- ... to meet
 - Software/System/Product → Feature, Package, & PPA requirements



Control & Programming Model

- CPU cores have been the heart of the programming model for a long time
 - Even accelerator SoCs like GPUs often use them as command/control processors
 - Less complex accelerator SoCs are driven via MMIO through interfaces like PCIe
- Processors/Embedded SoCs rely on the CPU to carry out control tasks
 - Direct interfacing with IPs over MMIO
- If the chip is meant to be end-user programmable, then the programming model (and associated tooling) is make-or-break for its success
 - Ex. Tiler in the mid-2000s vs. Nvidia today – Graphics and then Graph-based ML played a role in success.

IP Selection

- There are many kinds of IPs to choose from
 - Programmable core IPs like CPU/GPU/NPU
 - Miscellaneous accelerator units for specific tasks
 - video codec, encryption, color blending
 - I/O & analog IPs
 - Bus management IPs
- Choice depends on PPA requirements +
 - Bus/memory hierarchy compatibility
 - not all CPUs work with cache controls needed for multi-core
 - Bus wrappers may be required i.e to support an AHB core on an AXI bus
 - Technology compatibility
 - Some IPs are distributed as RTL. Some are technology-optimized.
- IP Exploration is completed with tools like Arm Socrates or Synopsys PA

Generators

- IPs do not usually ship as straight RTL or hard-macros → Generators are vendor-provided services or software that handle the parametrization and building of IP.
- Generators also often refer to more complex IP parametrization
 - [ucb-bar/gemmini: Berkeley's Spatial Array Generator](http://ucb-bar/gemmini)
- Generators are also a part of business model
 - IP generation can be tiered behind different licenses
 - SiFive's Core Designer is a web-based commercial IP generator – it can be freely accessed for purposes of seeing IP parameters.

SiFive Core Designer

Modes & ISA

On-Chip Memory

Ports

Security

Debug & Trace

Interrupts

Design For Test

Clocks and Reset

Other Default IPs

RTL Options

Ports

☒ Front Port ?

Pass through: Disabled

Edit Protocol & Bus Width

Done Editing

Protocol

AXI4

AHB_Lite

Note that transactions from an AXI Front Port are not able to target AHB/AHB_Lite Memory, System, or Peripheral Ports.

Width

32-bit

64-bit

128-bit

AXI ID Bit Width ?

4

8

16

Untitled E2 Core Core Complex

E2 SERIES CORE

RV32IMAFCB

Machine Mode • User Mode

Multiply (1 Cycle) • Atomics • FP (F)

Present

SCIE

Core interfaces = Separate

Clock Gating

1 Perf Counter

Adr. Remappers 16

PMP 4 Regions

µInstr Cache

None

TIM 0

32 KiB

TIM 1

32 KiB

Debug Module

APB • SBA

4 HW Breakpoints

0 Ext Triggers • CRA

CLIC

4 Configuration Bits

127 Interrupts

Front Port

32-bit AXI4

System Port 0

32-bit AHB_Lite

System Port 1

None

Peripheral Port

32-bit AHB_Lite

Base: [E21 Standard Core](#)

 ECE 260C SP26 Kahng

8

Memory Hierarchy & Bus/NoC

- Memory access is the key driver of performance in most general-purpose compute scenarios
 - OoO CPU design, GPU hardware-multithreading, etc. are all methodologies for hiding the latency of memory
- Memory interfaces must be chosen carefully
 - Internal SRAM or DRAM, external DRAM interfaces, host memory over PCIe, ...
- Cache hierarchy presents a large design space
 - How many levels of cache? Separation between instruction/data cache? ? System-level cache? Cache coherency requirements? Cache banking and connectivity
- Bus/managers must support bandwidth & coherency requirements
- NoCs see more adoption in newer networking/ML applications

I/O Integration

- Besides core signals and external memories, serial I/Os are often required
 - USB, Ethernet, PCIe, HDMI, SATA, Die-to-Die – Many IPs for these standards available
- “SerDes” architecture
 - SerDes units handle the multi-GHz requirements of modern serial protocols
 - Conversion between parallel bus data to serial output + Clock/Data Recovery
- I/O IPs are usually split into two parts
 - PHY – provides the technology-dependent SerDes and CDC requirements
 - MAC – implements the (packet) protocol required in technology-agnostic RTL
 - Often supporting DMA to read/write data directly from/to memory

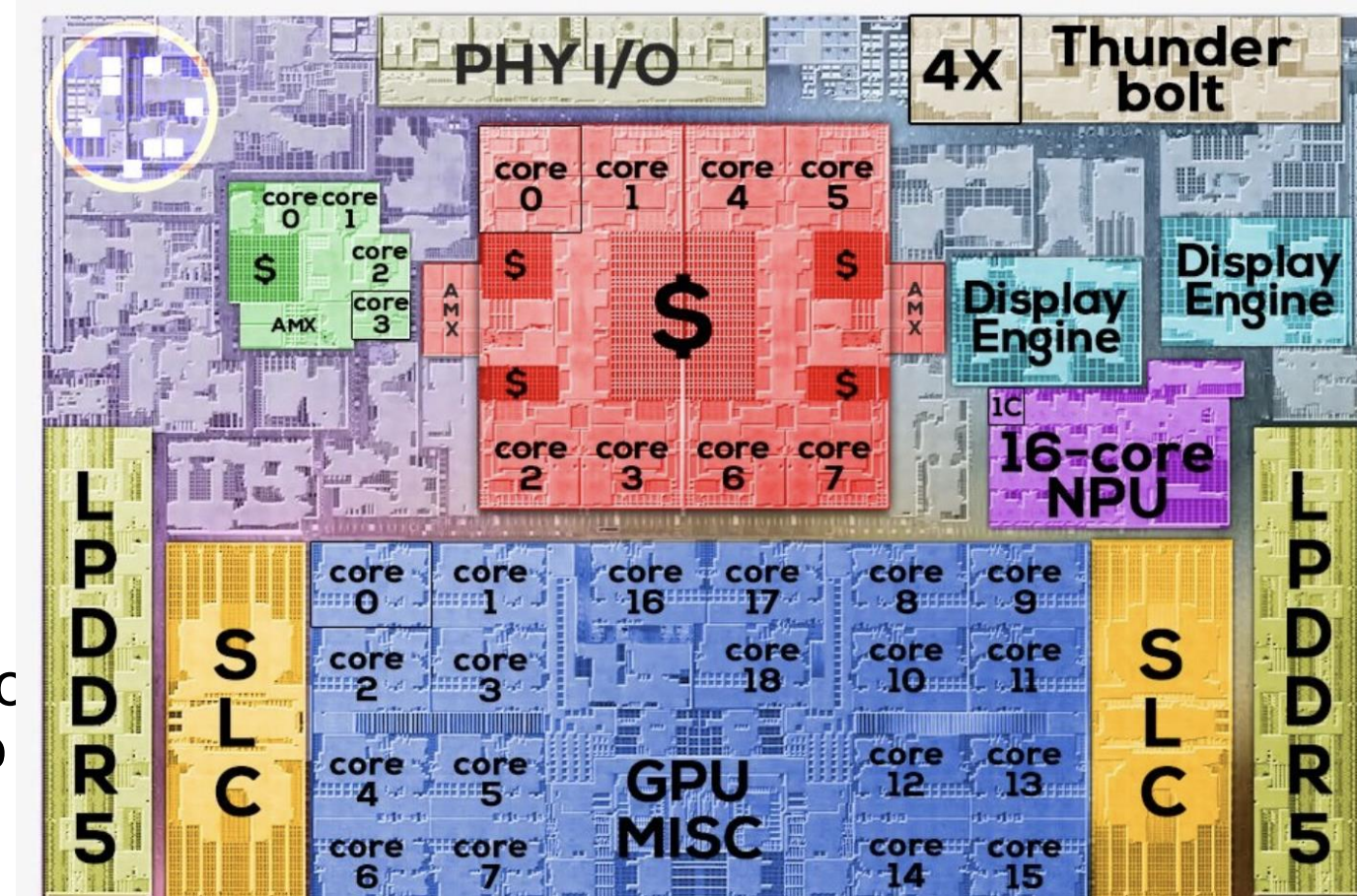
Physical Implementation

Agenda

- Architectural Decisions
 - Busses, Programming Model, IP Generators, I/O by protocol
- **Physical Implementation**
 - **SoC Floorplanning**
 - Design strategies
 - MIIMs and partitioning
 - Yield Recovery
 - Dataflow
 - Core Floorplanning
 - Technology Mapping
 - Clock & Power Planning
 - Logic Implementation
 - Miniproject 2

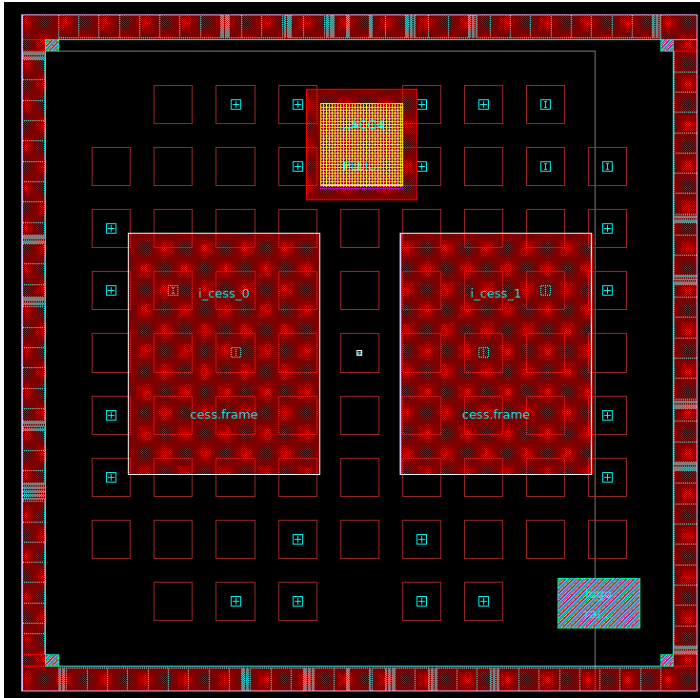
SoC Floorplanning

- Floorplan is crucial to performance
 - Greater Bus Lengths → Repeaters/Cache Structures
 - Latencies affect stall time → Lower Performance/Greater Power usage
- Thermal/packageing affect placement
 - DVFS cores slow down if too hot
a core cluster needs spacing to disperse heat.
 - I/O placements vs. package

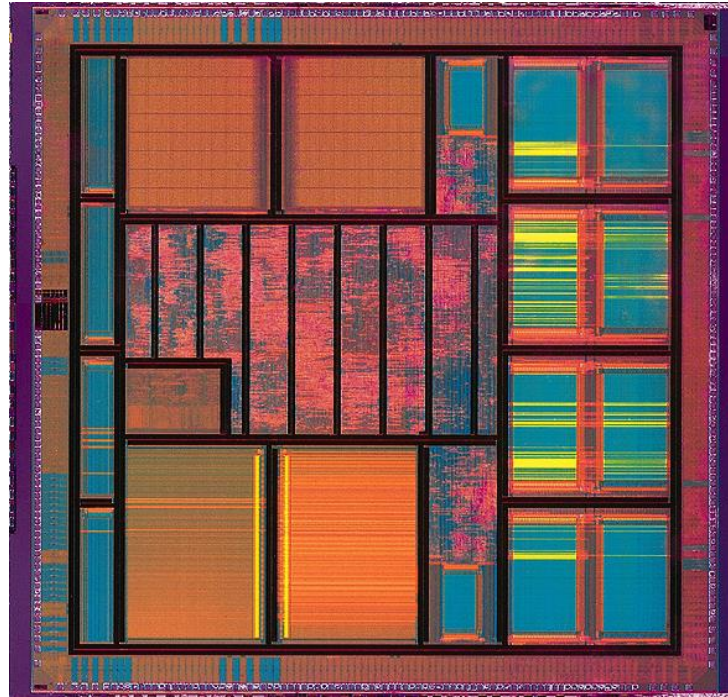


Top-Level Design Styles

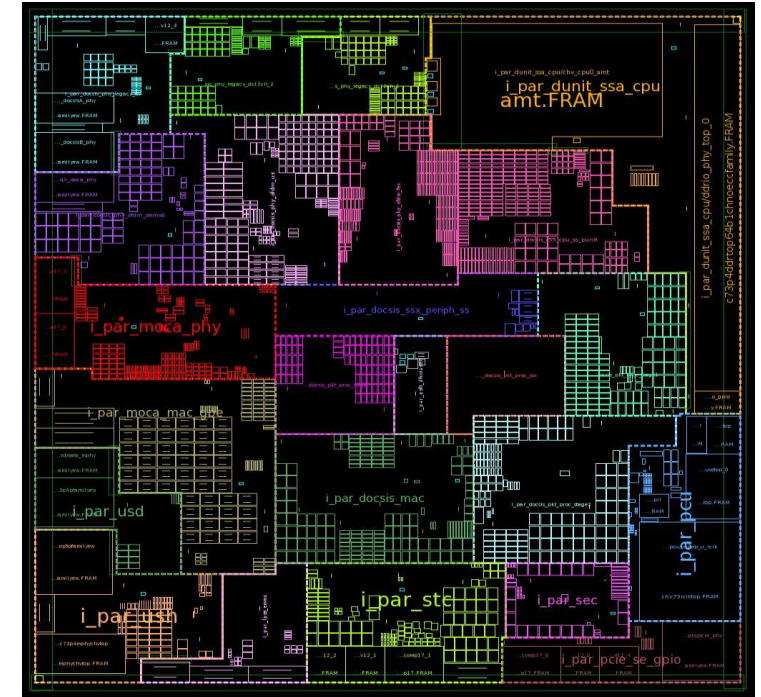
Channel-based



Narrow-channel



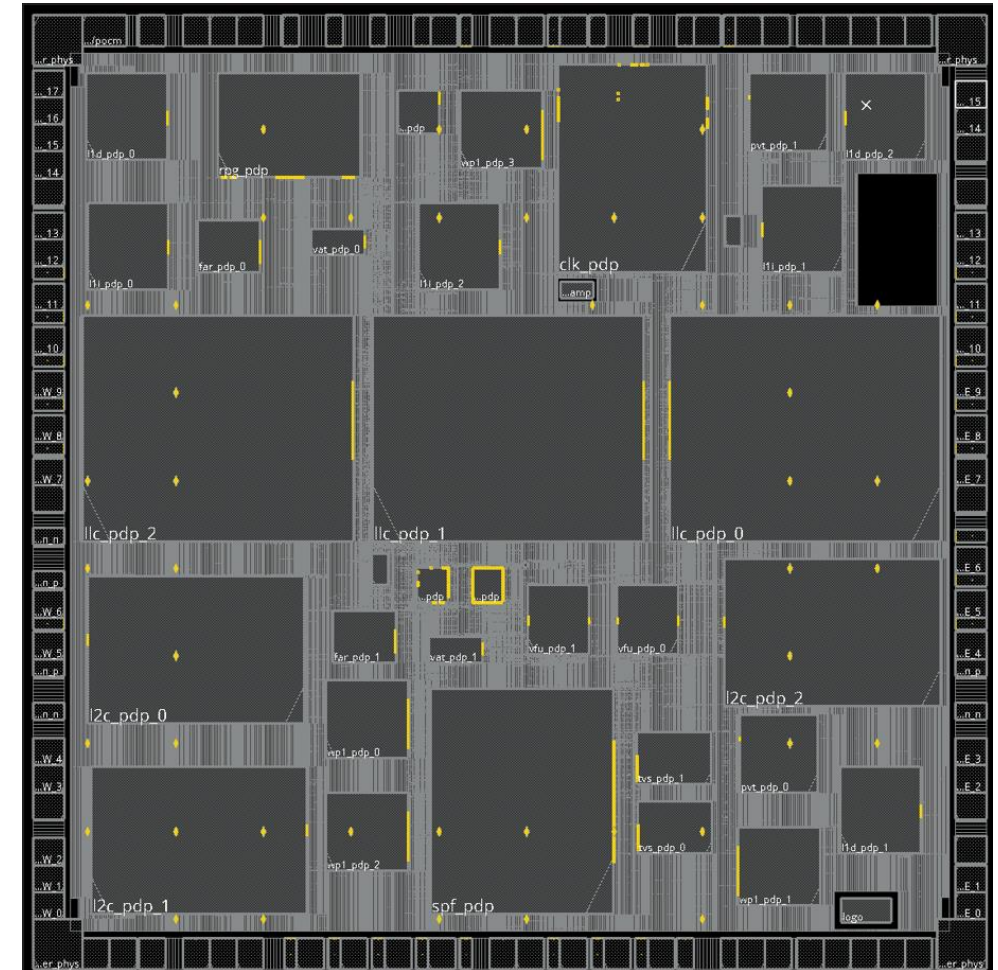
Fully-abutted



<https://en.m.wikipedia.org/wiki/File:Diopsis.jpg>

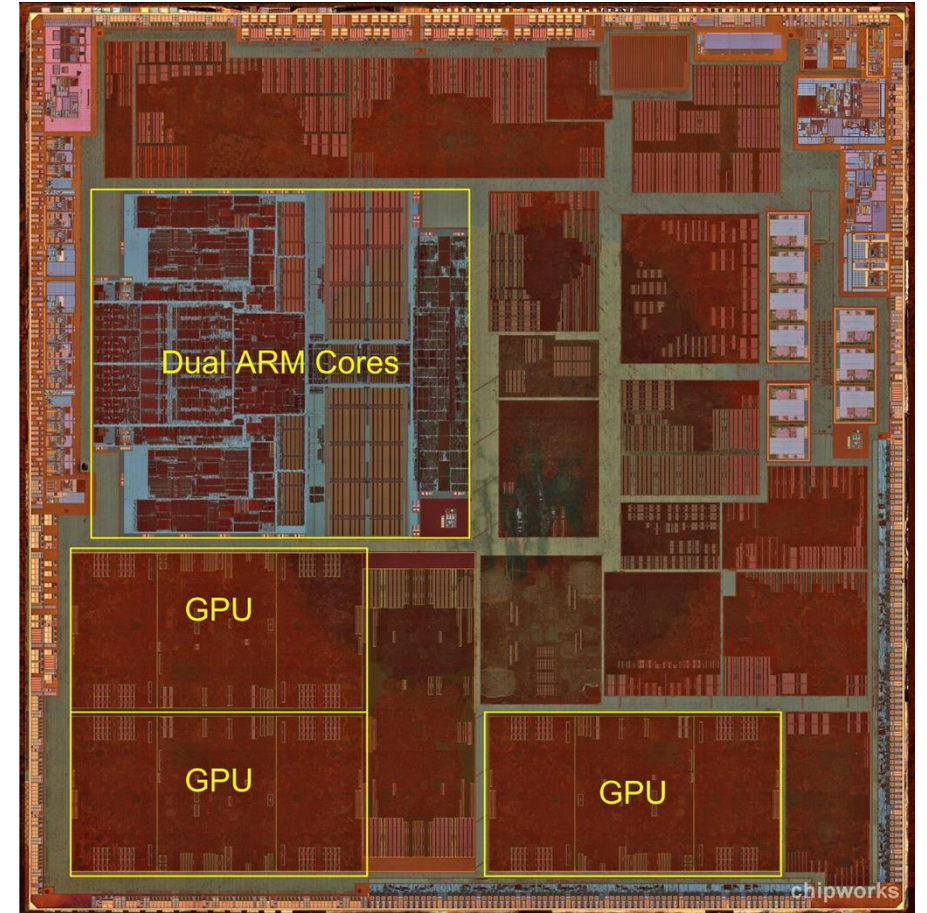
Channel-based

- Bottom-up approach, child blocks and macros dictate their shape/size.
- Ideal for test chips/prototypes where die area is not a concern and top-level area and logic is can be easily managed.
- Pushdown information is minimal.
 - Topmost layer
 - Rough pin locations: side, region



Narrow-channel

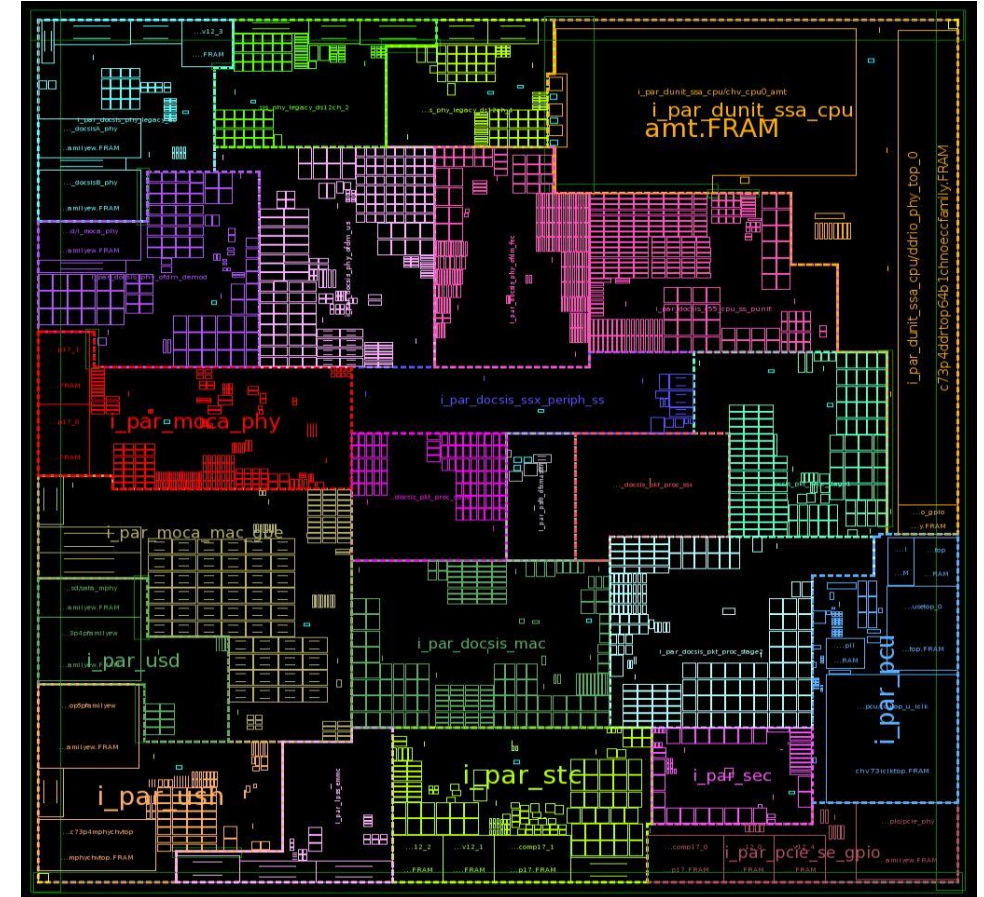
- Most partitions are abutted, but small channels are left for clock routes, DFT (design for test), and high-speed interfaces.
- Avoids fly-over routing and feedthroughs
- Keeps placeable area small for larger dies
- Auto place and route tools are not designed to handle channels, requires hand-holding or custom placement and routing.
- Creates large perimeters that must be bounded/capped to avoid DRC violations.
- Latency balancing is impossible without backtracking, which costs power.



Apple A6, Photo & Annotations from Chipworks

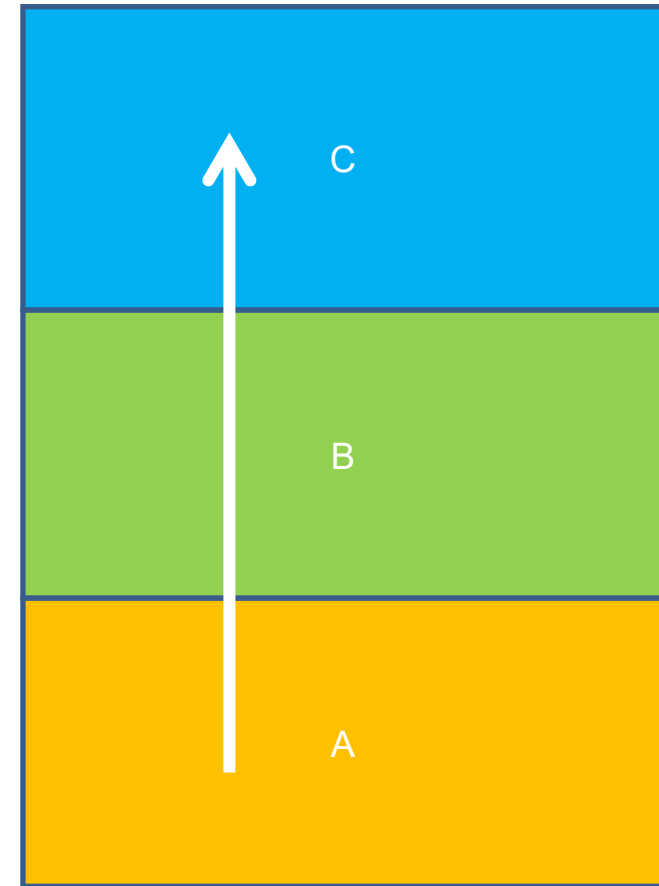
Fully-abutted

- No top-level area exists. Each partition is resized to abut on all sides with neighboring partition.
- No wasted die area
- **No top-level place and route is required**
- Forces rectilinear shapes
- Feedthroughs are unavoidable
- Clock routes and critical interfaces must be pushed into the partitions

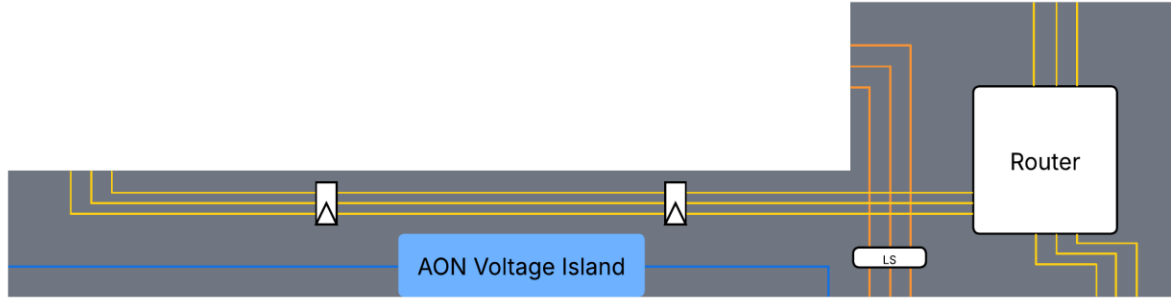


Feedthroughs

- Signals must travel from one partition to another by way of one or more partitions
- The feedthroughs modify the partitions by adding ports, routes, buffers, possibly flop repeaters.
- If blocks are converted to partitions, then the module has to be modified to include the feedthroughs.

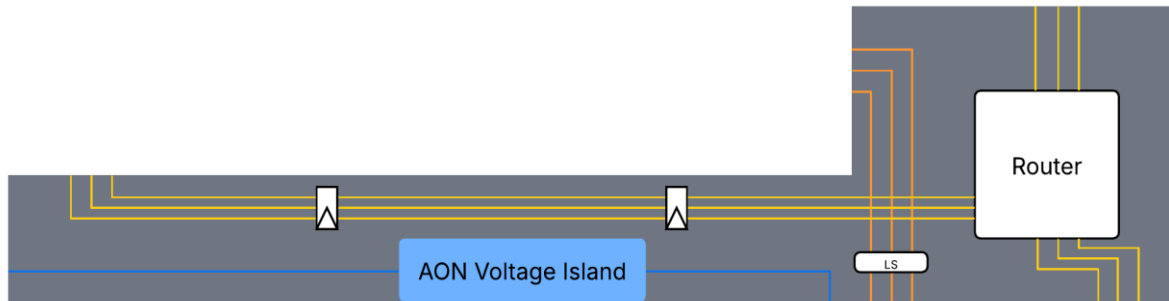


Channel Partitions



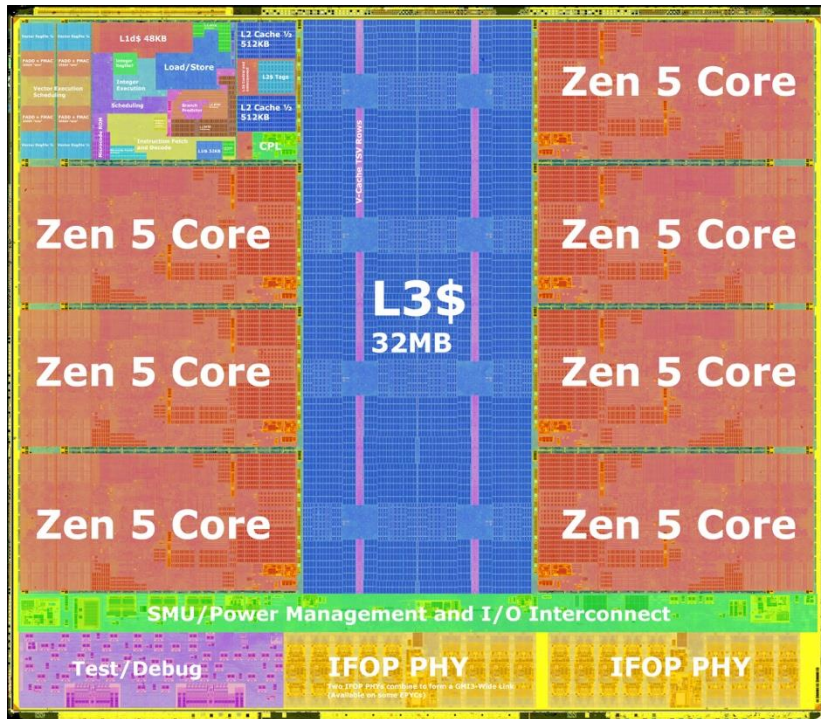
- Since fully-abutted designs contain no top-level logic, need to create channel partitions
- Channel partitions often contain:
 - Repeater logic: flops, buffers and inverters
 - Router blocks
 - Level shifters and isolation cells
 - Voltage islands
- Perform repeater studies and evaluate delay per mm
 - Evaluate different metal layers, non-default routes (NDRs), repeaters
- Split top-level channels into simple partitions
- Assess port locations, especially for busses.
- Design & select NDRs to use for different types of signals
- Plan flop count and locations

Channel Partition Implementation

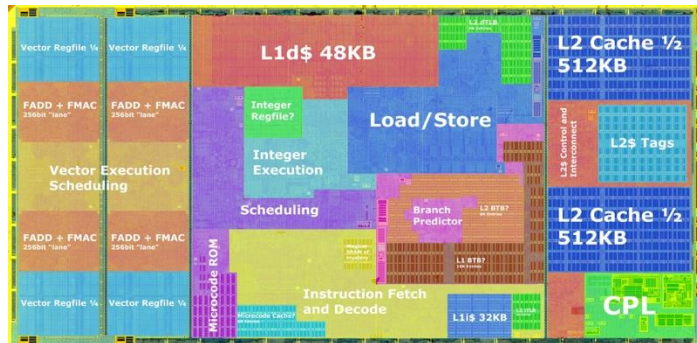


- Assign an NDR and layer list to every bus or signal
- Order busses, place ports
- Place flops and voltage islands
- Custom bus routing
- DRC Cleanup

Multiply-Instantiated Modules (Structural Reuse)



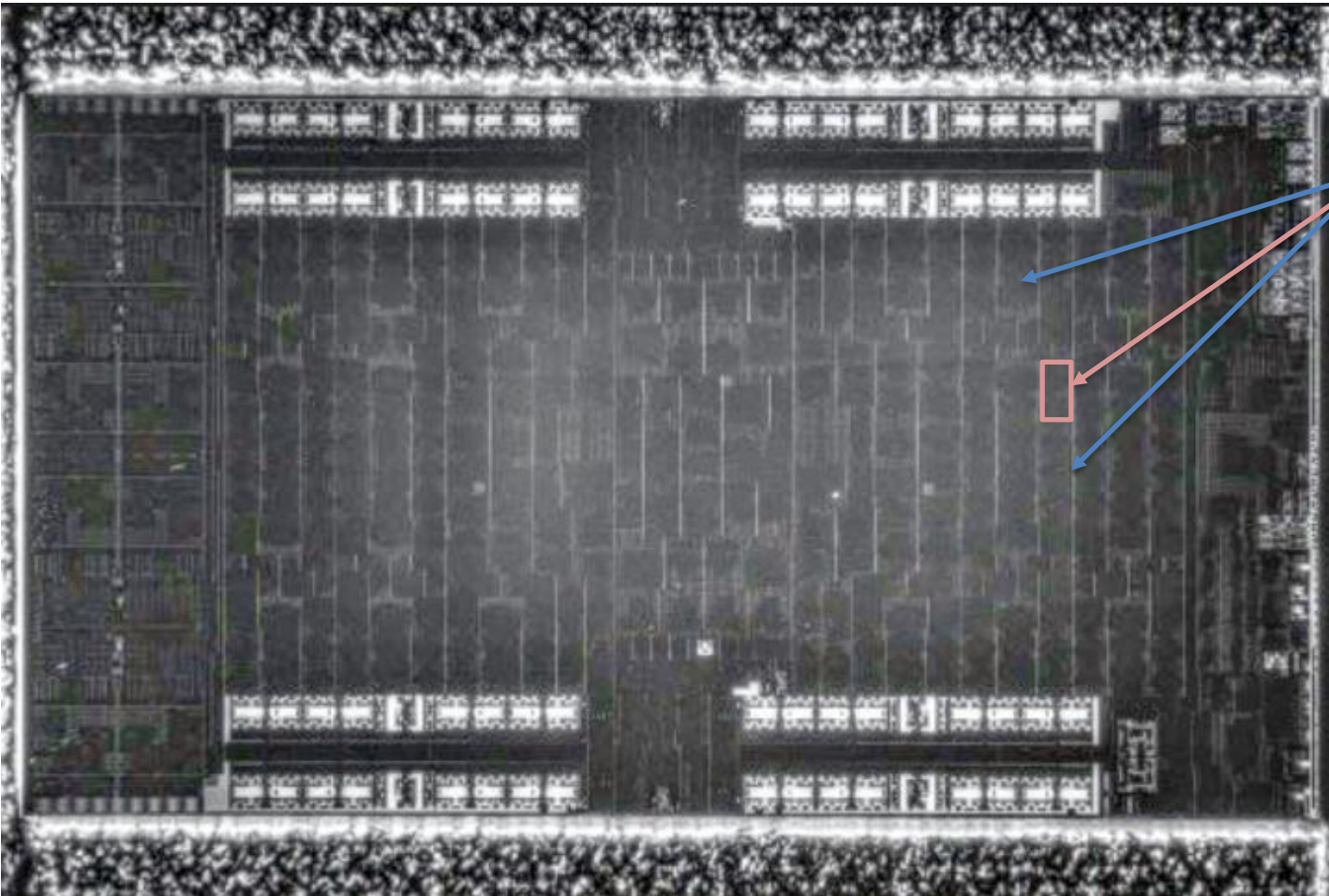
- Hardening repeated logic has many benefits
 - Reduce PD tool turn-around time, accelerate schedule
 - Large power and timing improvements across the chip
 - Can use structured or semi-custom methods at small scales
 - Re-use the same block across subsequent chips



- Straightforward choice in simple cases, allows small teams to build large, powerful chips

Source: Nemez on X

Example: PS5 Design Partitioning

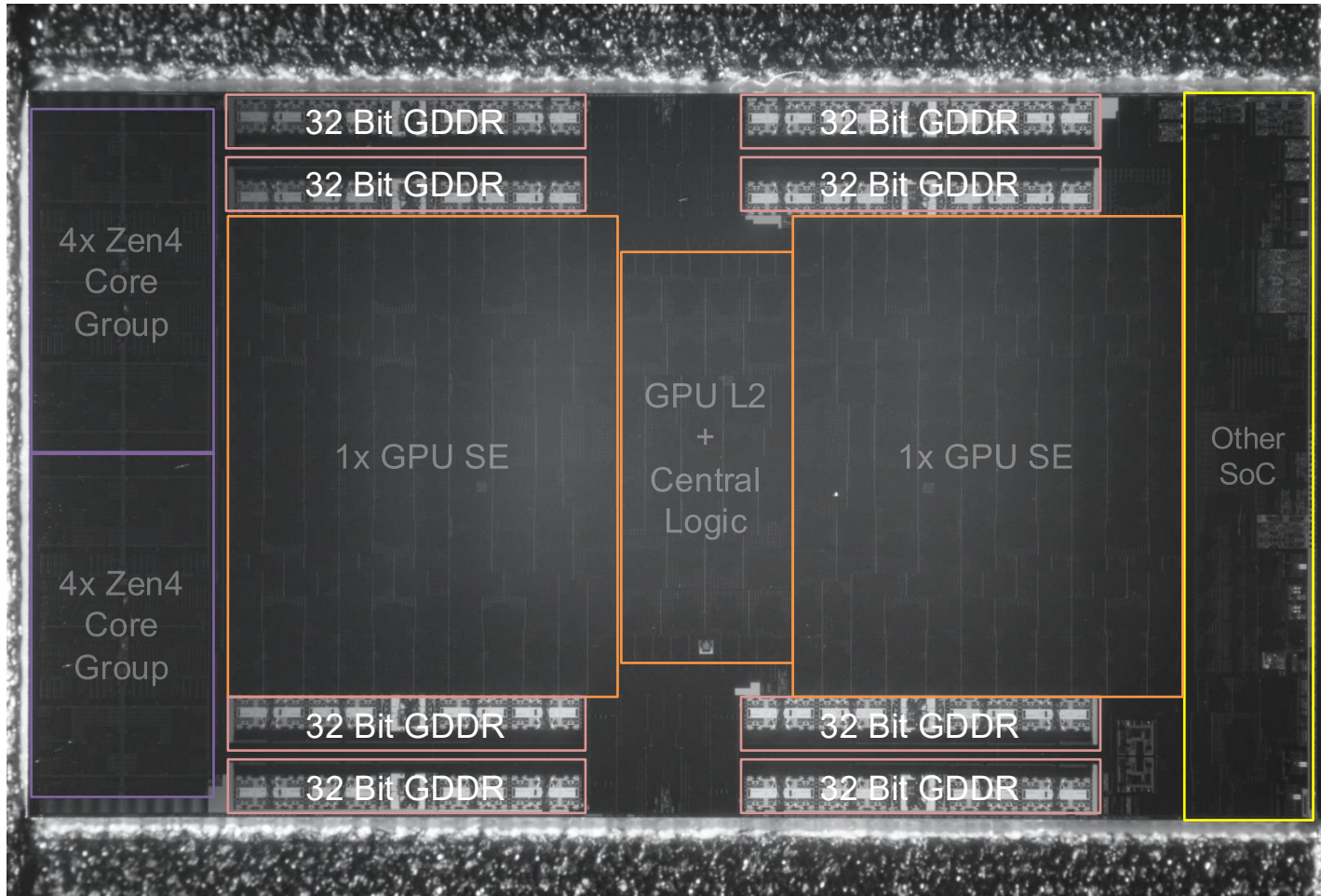


Enhancing the contrast ratio allows the boundaries of structural design partitions to be seen

The PS5 SoC is broken into several hundred of them (about 100-300k μm^2 each) to reduce the time to convergence of quality routes

Die Shot Source: <https://www.notebookcheck.net/PlayStation-5-die-shots-confirm-no-AMD-Infinity-Cache-support-other-RDNA-features-may-been-out-too/520747.0.html>

Example: PS5 Design Partitioning in Detail

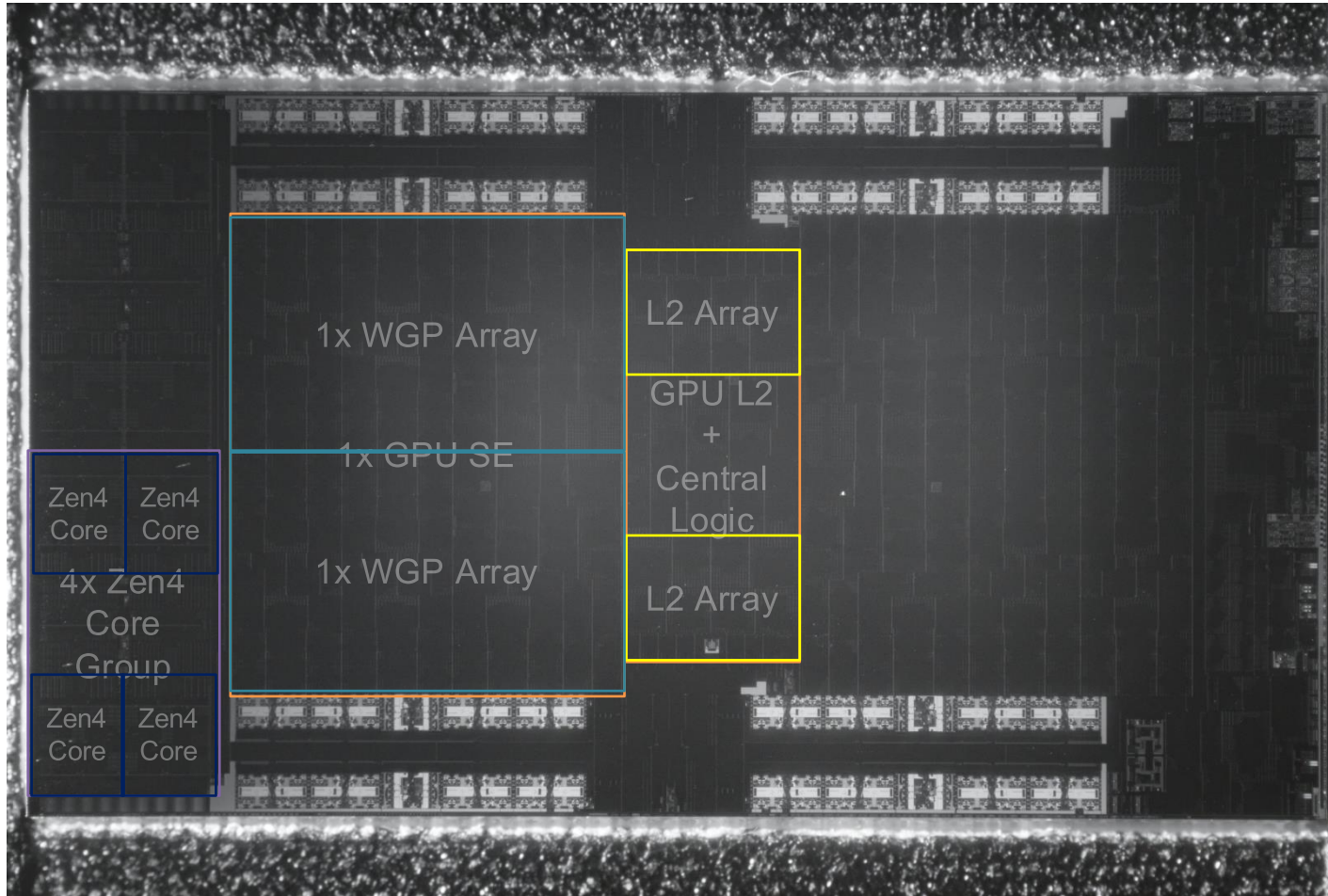


Looking closely at the design there is several repeated sections:

- Each Zen4 Core Group is exactly the same
- Every GDDR Phy + Controller is exactly the same
- The GPU SEs are mirror images of each other

This reuse allows them to "copy and paste" the work they did to synthesize/layout these blocks drastically reducing design time.

Example: PS5 Design Partitioning Level 2

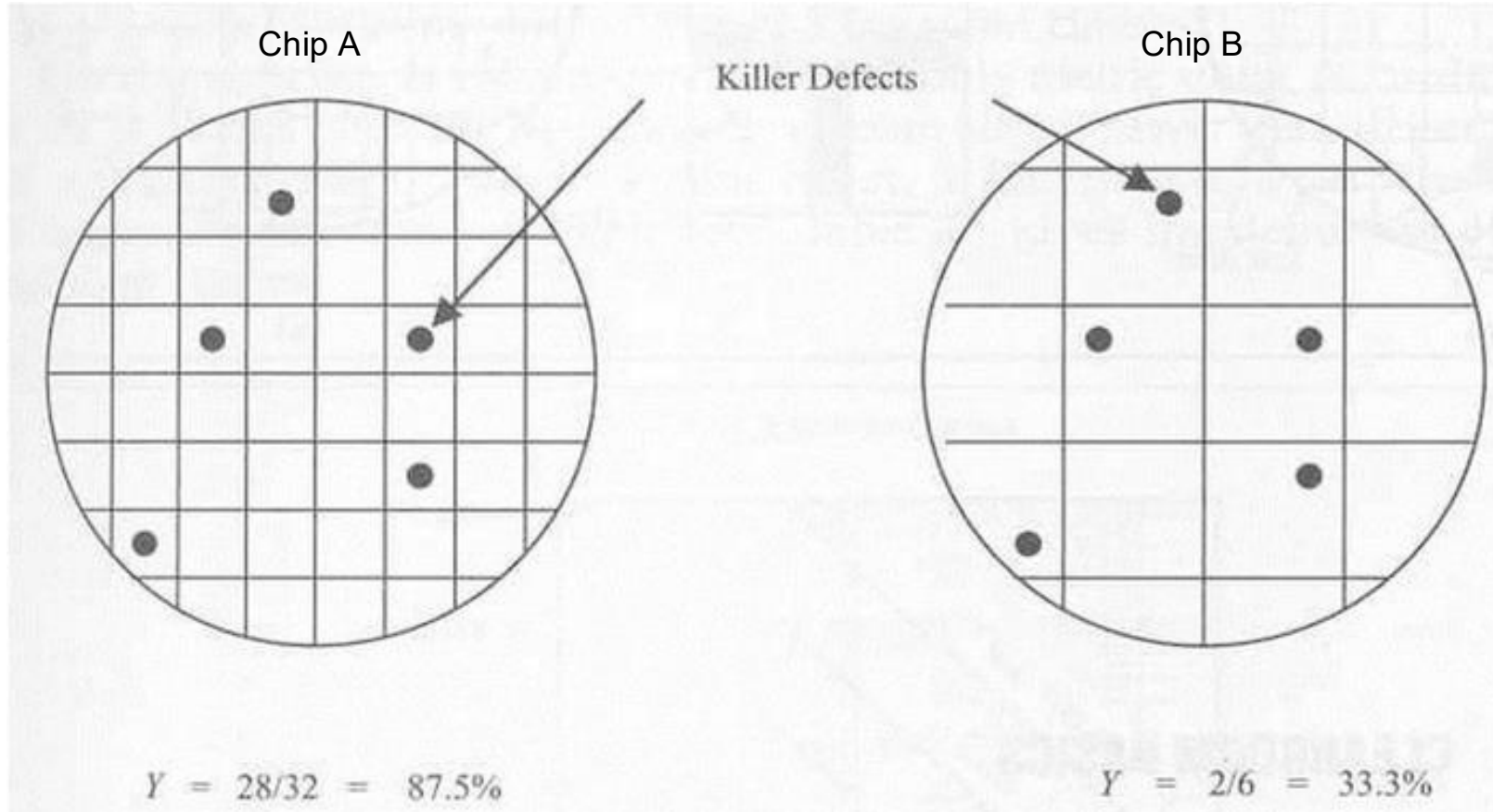


They go further, by nesting their reuse:

- A **WGP array** is mirrored inside of each SE
- The top and bottom **L2 arrays** are mirrored
- There is four mirrored copies of the **Zen4 core** in each core group.

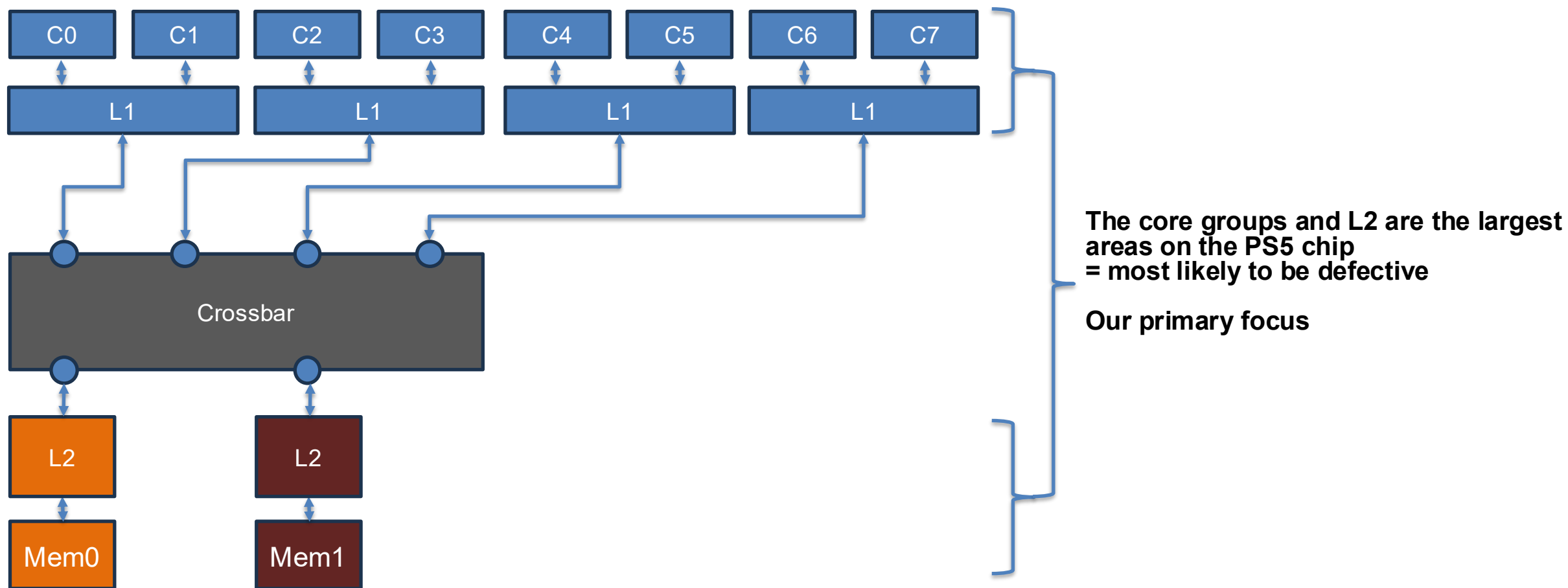
Die Shot Source: <https://www.notebookcheck.net/PlayStation-5-die-shots-confirm-no-AMD-Infinity-Cache-support-other-RDNA-features-may-been-cut-too520747.0.html>

Manufacturing Defects

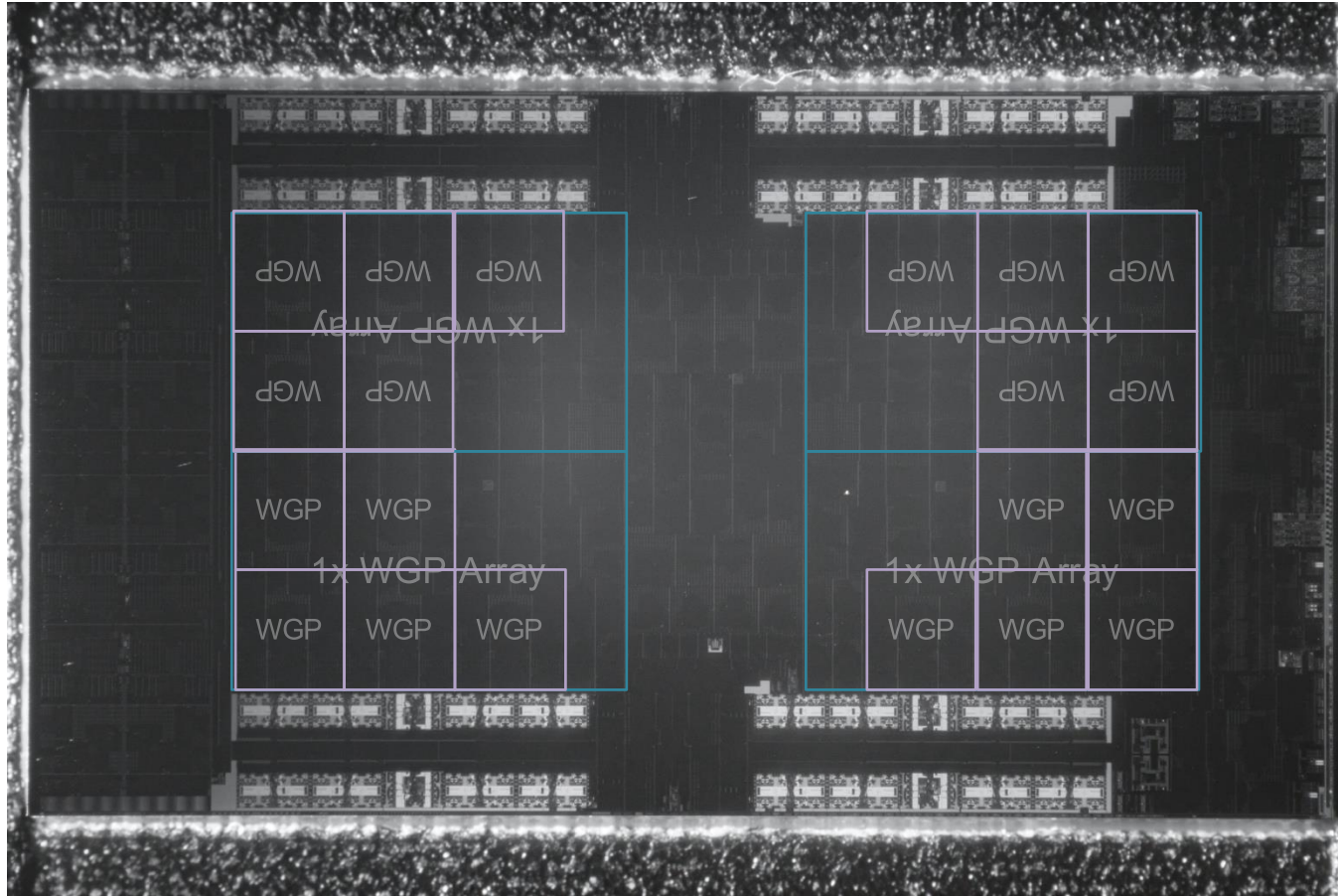


- Chips are real objects, defects during manufacture are unavoidable
- Defects in a chip kill it unless you can design to tolerate them
- Lost chips due to defects increase the price per mm^2 of each chip
- Chip B costs $>10\times$ of Chip A:
 - $4\times$ the cost comes from B being $4\times$ the size
 - $>2.5\times$ comes from the fact Chip B is much less likely to be defect free
- Yield recovery strategies try to make a defective chip look like a working one so it can still be sold

Yield Recovery



Example: PS5 Yield Recovery



Inside a WGP Array there is 5x exact copies of a WGP

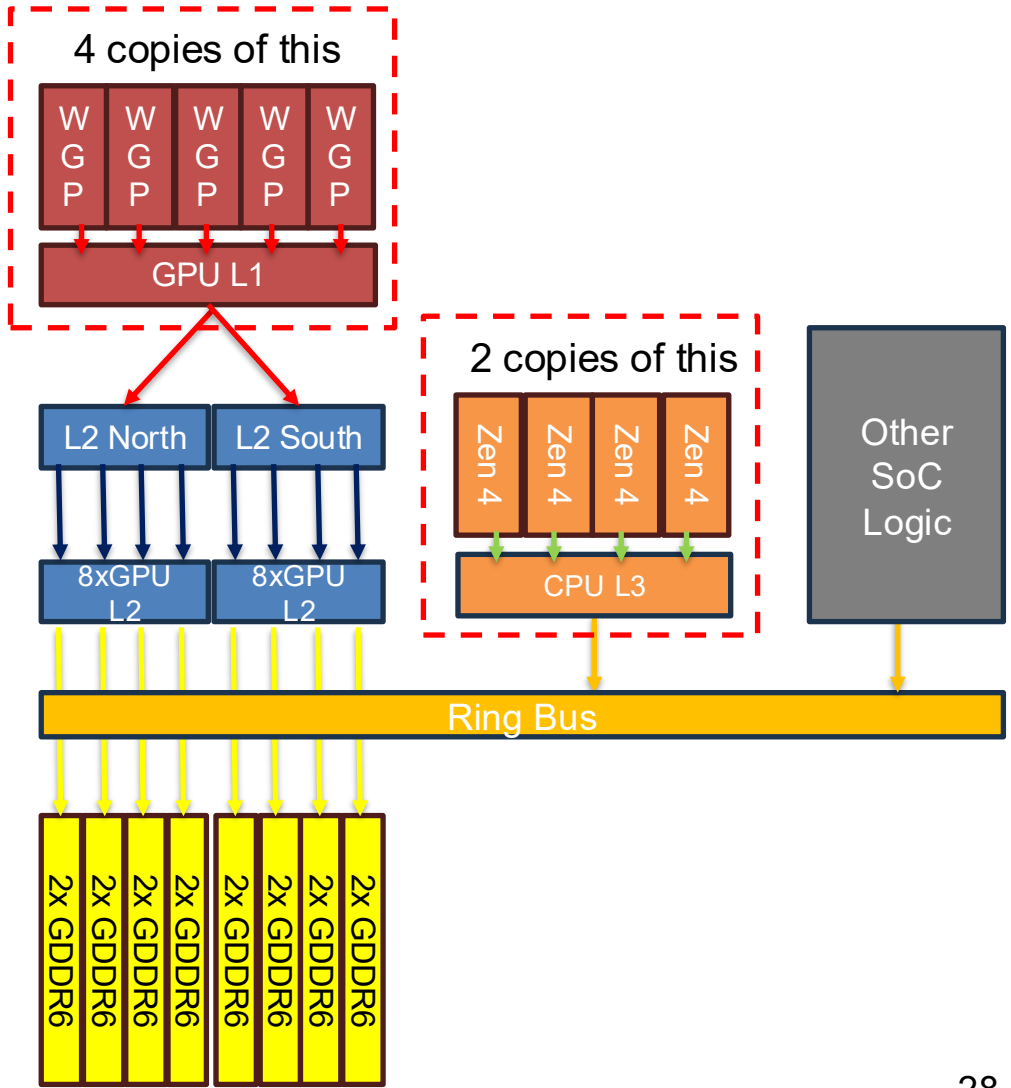
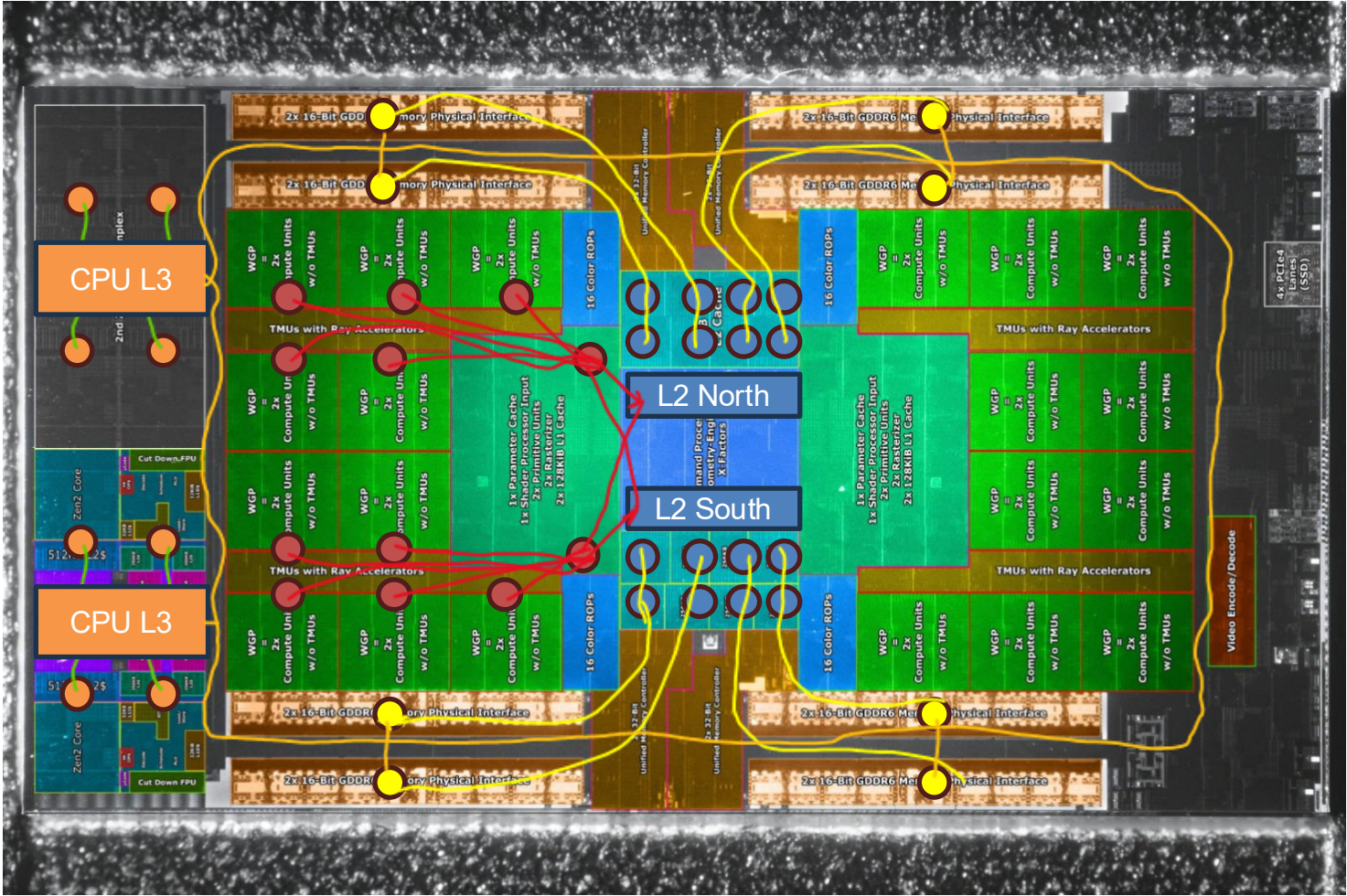
This allows a very high quality synthesis of the WGP, as the work gets reused 20 times on the chip.

You can also see some of the yield recovery mechanisms here, as the part only has 18 WGPs logically, but has two redundant WGPs physically.

Die Shot Source: <https://www.notebookcheck.net/PlayStation-5-die-shots-confirm-no-AMD-Infinity-Cache-support-other-RDNA-features-may-been-out-too/520747.0.html>

Memory-driven Dataflow

Likely dataflows from floorplan

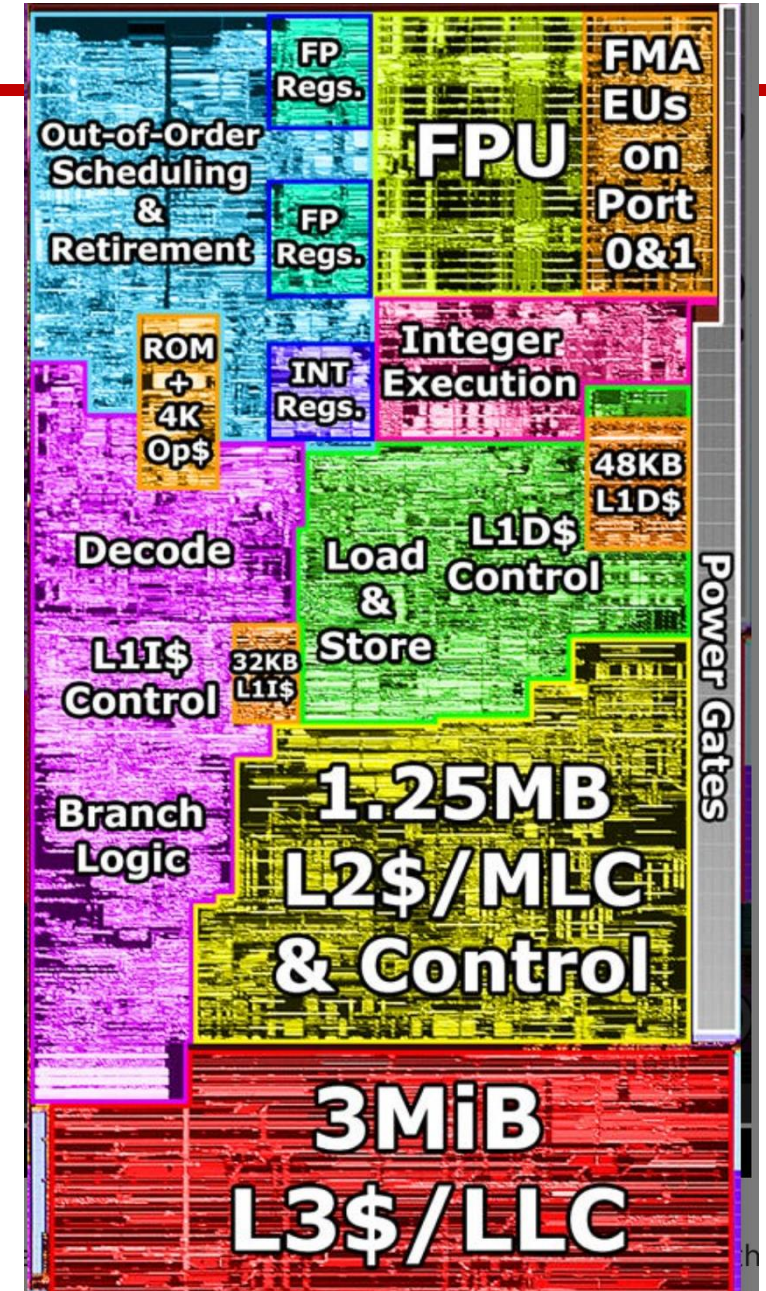


Agenda

- Architectural Decisions
 - Busses, Programming Model, IP Generators, I/O by protocol
- Physical Implementation
 - SoC Floorplanning
 - Design strategies
 - MIIMs and partitioning
 - Yield Recovery
 - Dataflow
 - **Core Floorplanning**
 - Clock & Power Planning
 - Technology Mapping
 - Logic Implementation
 - Miniproject 2

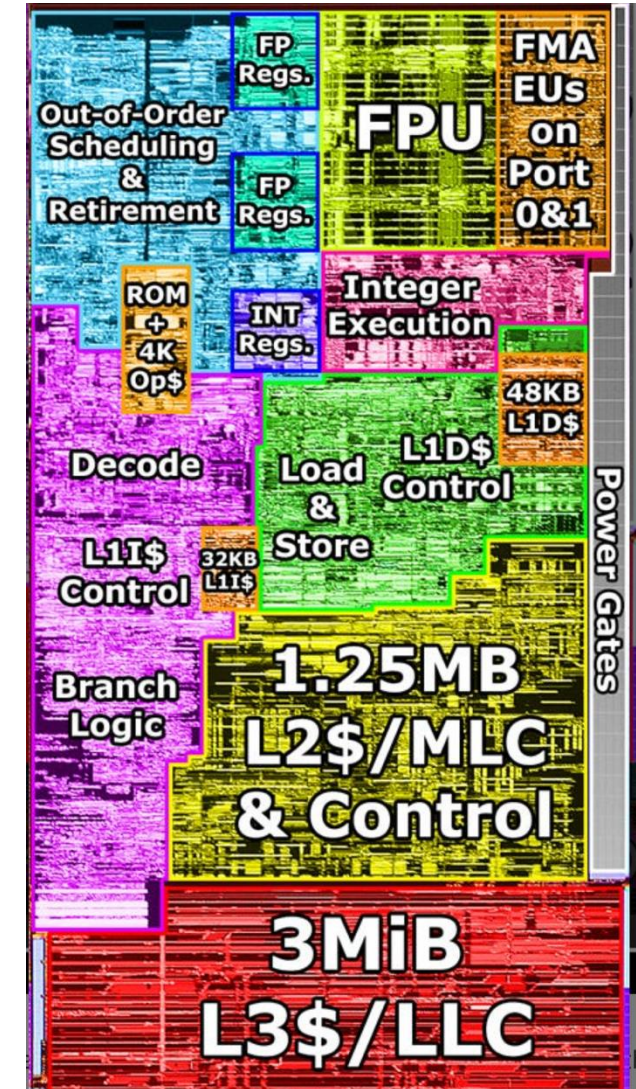
Core Floorplanning

- Must fit within SoC floorplan
 - Best: fixed size/AR for all instances of a core
- Architectural Blocks $\neq$ Physical Blocks
- Dataflow
- Proximity of memories/RFs to logic
- Mapping of memories to technology
- Clock & Power requirements
- Many logic implementation methods



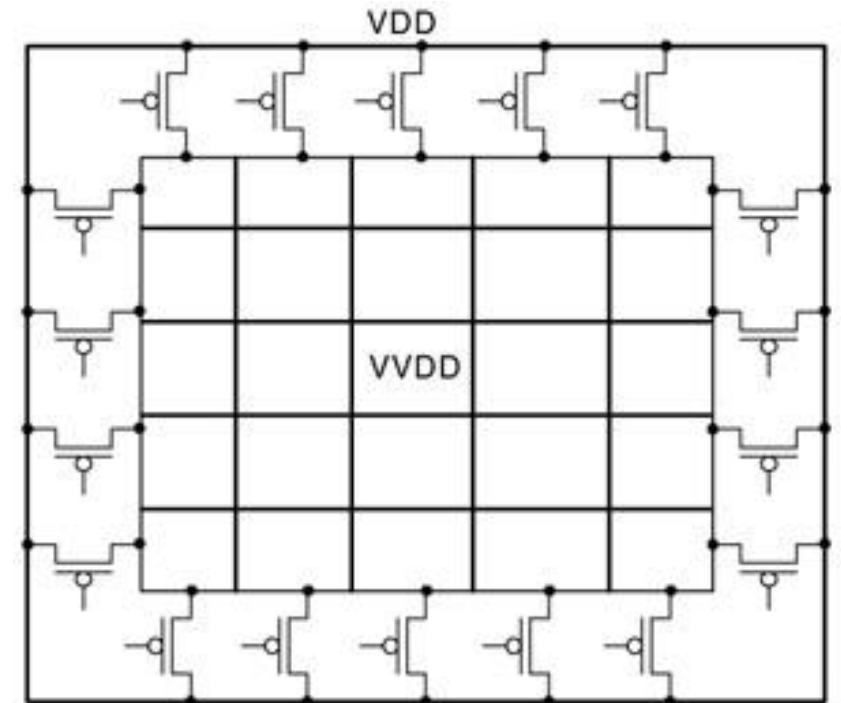
Clock & Power Planning

- Power efficiency is of increasing importance
 - Mobile applications, Dennard scaling breaks down
- DVFS and Clock Gating introduce complexities
 - Clock domain crossing, reset requirements, thermal monitors, etc.
- Power gating is even more expensive
 - Power gates are large, isolation/retention cells are required
- Programming Model
 - How will these features be configured? Drivers? Control curves? Hardware FSMs.
 - Standards like SDC and UPF
- Homogenous core clustering is a common solution



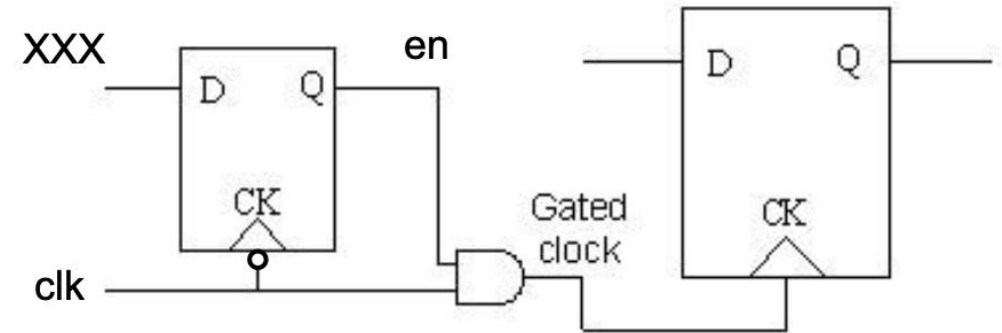
Low-Power Implementation vs. Architecture

- Very few “invisible” or simple knobs
 - Gate resizing, static Vt library
- Some may need more PD support
 - Multi-Vt design
 - Clock/Power Gating
 - Multi-voltage/Dynamic Voltage-Frequency Scaling
- ...and architectural/software support
 - Clock/Power Gating, DVFS



Clock Gating & Scaling

- Architectural considerations
 - Clock gates need to be hardened in the RTL
 - Control needs architectural support
 - Clock scaling needs adequate CDC
 - Glitch prevention for I/Os
- Physical Design Requirements
 - [STA support](#) – create_generated_clock
 - On-chip clock scaling - macros for PLLs must be placed



Power Gating

- Power Gating requirements
 - Power gates – [Header/footer cells](#), mother-daughter
 - Isolation cells
 - I/Os need to be disconnected from wider design, else floating
 - ANDs that zero I/Os interfacing with gated block
- Careful design against inrush current

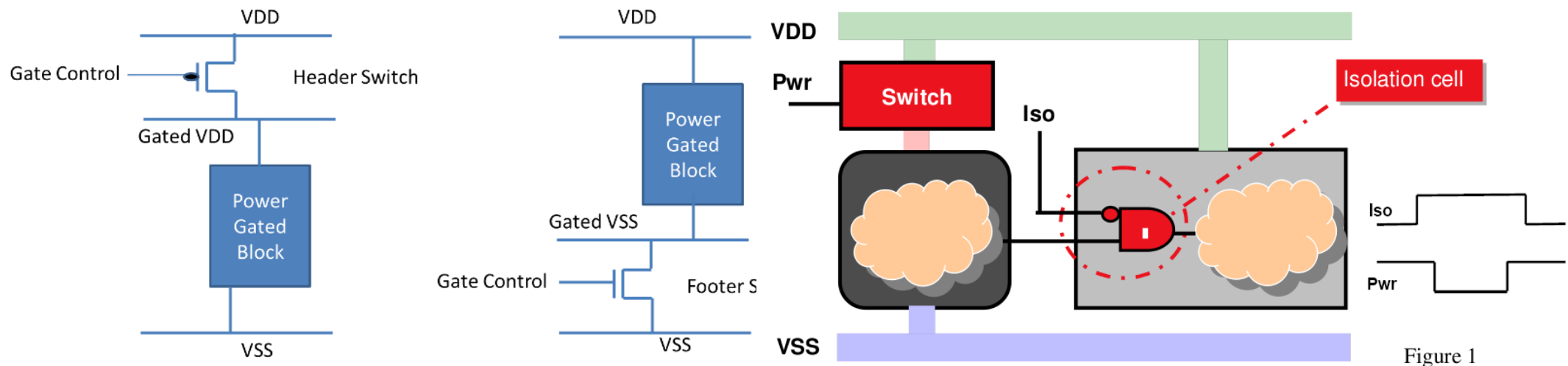


Figure 1

Technology Mapping

- Technology-agnostic design is the dream
- Reality: we need to plan w.r.t.
 - Core sizes (dependent on PDK cells)
 - PDK SRAMs or compiled
 - Primitives for clock and power
- But, designs can be agnostic on the RTL level
 - Wrappers for:
 - non-inferable elements like SRAMs or primitives/mixed IP like PLLs or isolation cells
 - Partition feedthroughs
 - Separation of concerns between different IPs as well as between I/Os is helpful

Division of Labor

- SoC implementation benefits strongly from the design's necessary hierarchy
- Commonly, Separate teams for in-house IPs
 - Major teams focus on one block like the CPU. Some teams work on multiple.
- Small blocks are handed off to a “block owner” that will handle the physical design, working with RTL implementation engineers
 - One person can own multiple blocks, depending on the scale of the blocks and the project
- A top-level design team works to define and refine the floorplan
 - They perform inter-IP integration, checking in with block owners.

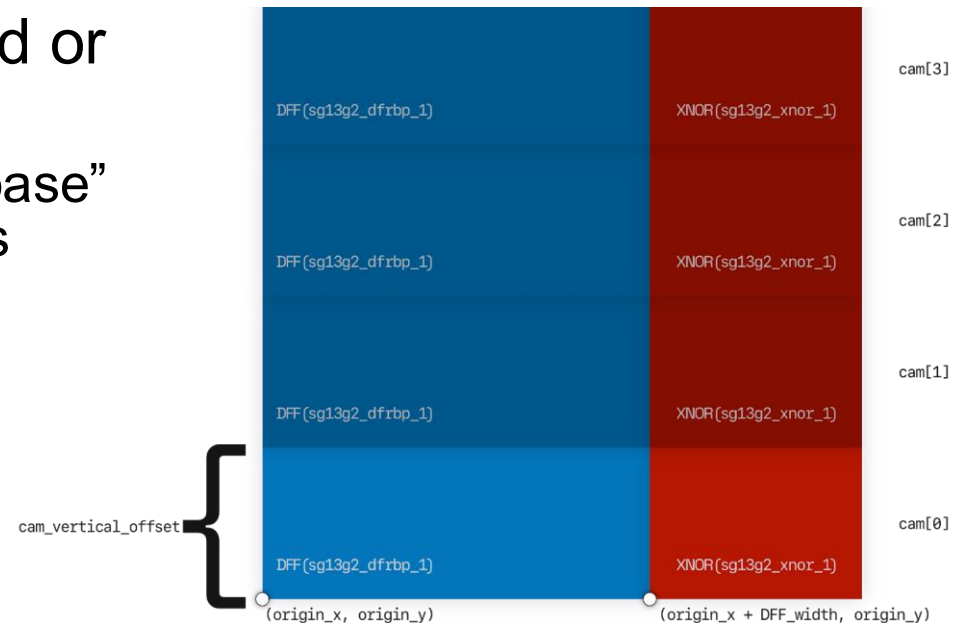
Logic Implementation

Agenda

- Architectural Decisions
 - Busses, Programming Model, IP Generators, I/O by protocol
- Physical Implementation
 - SoC Floorplanning
 - Design strategies
 - MIIMs and partitioning
 - Yield Recovery
 - Dataflow
 - Core Floorplanning
 - Technology Mapping
 - Clock & Power Planning
 - **Logic Implementation**
 - Miniproject 2

Structured Datapaths

- Highly regular manual/scripted placements of repeating elements
 - In Lab 2, you did this on individual standard cells but this can be done with small blocks too
- PPA benefits in aligning physical layout with dataflow
 - Tight packing can improve area efficiency especially when repeated elements are tightly optimized themselves (i.e. using high density semicustom adders).
- Costly – Labor intensive and affects agnosticism
 - Tool improvements continue to offset the need or change the scale of elements being placed.
 - Intel in 2024: “Lion Cove: Modernizing Design Database”
Methodology changed from small ~10k cell partitions to 10-100x that.



Retiming & Repipelining

- Retiming
 - a synthesis optimization that moves logic around sequential element boundaries to balance pipeline stages, minimizing the critical path.
 - ABC/yosys support it
- Repipelining is the application of retiming (either automatic or by hand), from an architectural point of view
 - As scaling breaks down and architectural advancements become harder to achieve alongside, it has become an increasingly useful tool, allowing higher clock speeds.
- Repipelining has costs
 - Implementation – needs hand-optimization of pipelines or DSE-based retiming.
 - Performance & Power – more pipeline stages = higher latency = more energy usage

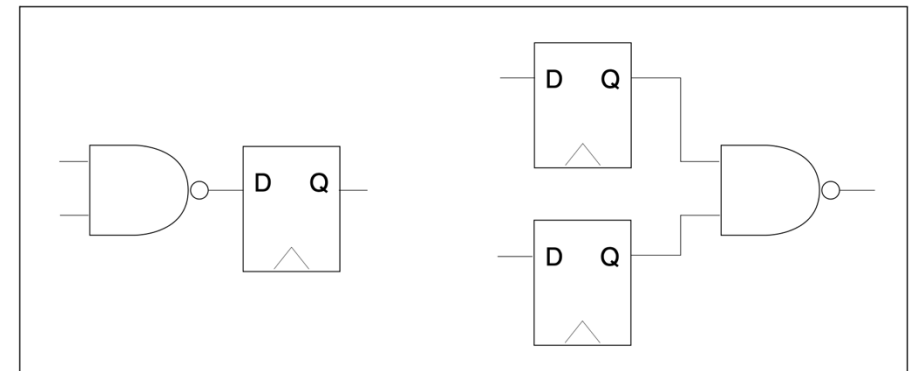


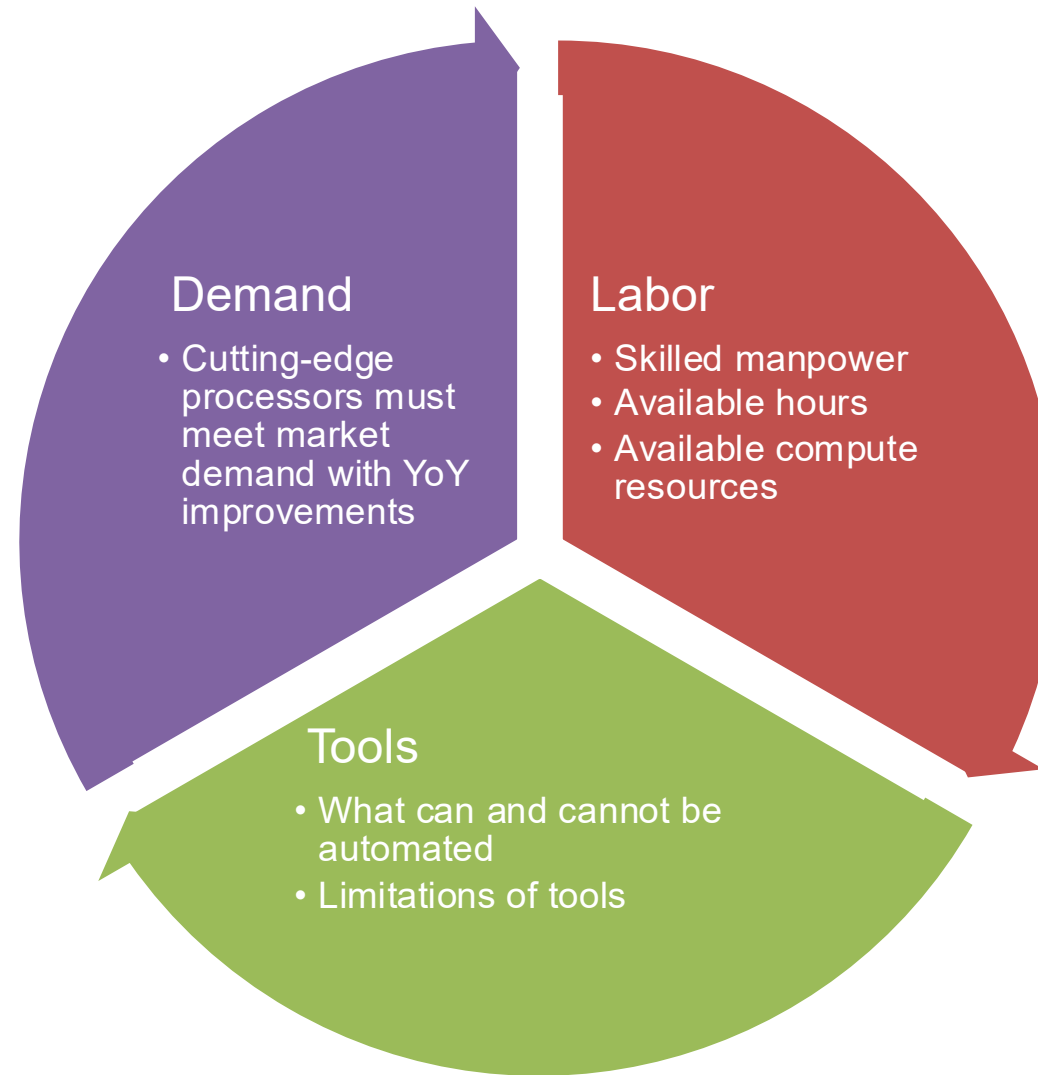
Figure 5-37 Retiming preserves combinational function.

High-Level Synthesis

- Standard RTL & synthesis can be clunky
 - Designing balanced pipelines
 - Interfacing with memories, busses, arithmetical units.
- HLSes seek to abstract these away
 - Familiar computer programming languages
 - Automated repipelining that determines stage count
 - Useful abstractions i.e. "Channel" generators to abstract bus requests into function calls
- Already sees commercial use
 - i.e. in codec and networking – saves time between modeling and implementation
 - AMD Vitis is freely available
- Open-source HLS is expanding – [Google XLS](#), [PNNL SODA](#)

```
#pragma hls_top
int test_unroll(int x) {
    int ret = 0;
    #pragma hls_unroll yes
    for(int i=0;i<32;++i) {
        ret += x * i;
    }
    return ret;
}
```


Implementation as a balance



Mini Project 2

Augmenting an SoC

- In teams of 2, you will make improvements to Croc, an SoC originally designed at ETH Zurich
 - Optimizations will be to PPA
 - You will submit your optimized design in one repo
 - Alongside any WIP or failed optimizations
 - For completed optimizations, you will also submit reports
 - You should submit at least one DRC-clean run of the flow
 - You will submit a report with
 - The optimizations you chose and justifications thereof
 - Implementations decisions and challenges
 - You should analyze improvements made (PPA needs reports)
- A full Project doc will be available soon after lecture

Optimizations

1. Hierarchical Floorplanning
2. Design Parametrization & DSE
3. Core: Optimized Multipliers & Adders
4. Core: Optimized Datapaths using SDPs
5. CTS Optimizations

The doc will have more details for all of these.

MP2 Submission

- Due June 9 – 11:59 PM
- Repo
 - All optimizations in source, all reports neatly organized
 - At least one DRC-clean run must be present
- Report
 - You will discuss all optimizations you attempted
 - Design decisions, justifications, challenges
 - Pointers to your code and reports
 - If you completed them, you will discuss and analyze improvements
 - Pointers to report
 - You will discuss division of labor and how you split the work

Main components of Croc

❑ Processor core (CVE2)

- ❑ The brain of the system

❑ Interconnect (OBI)

- ❑ Tying things together

❑ Memory

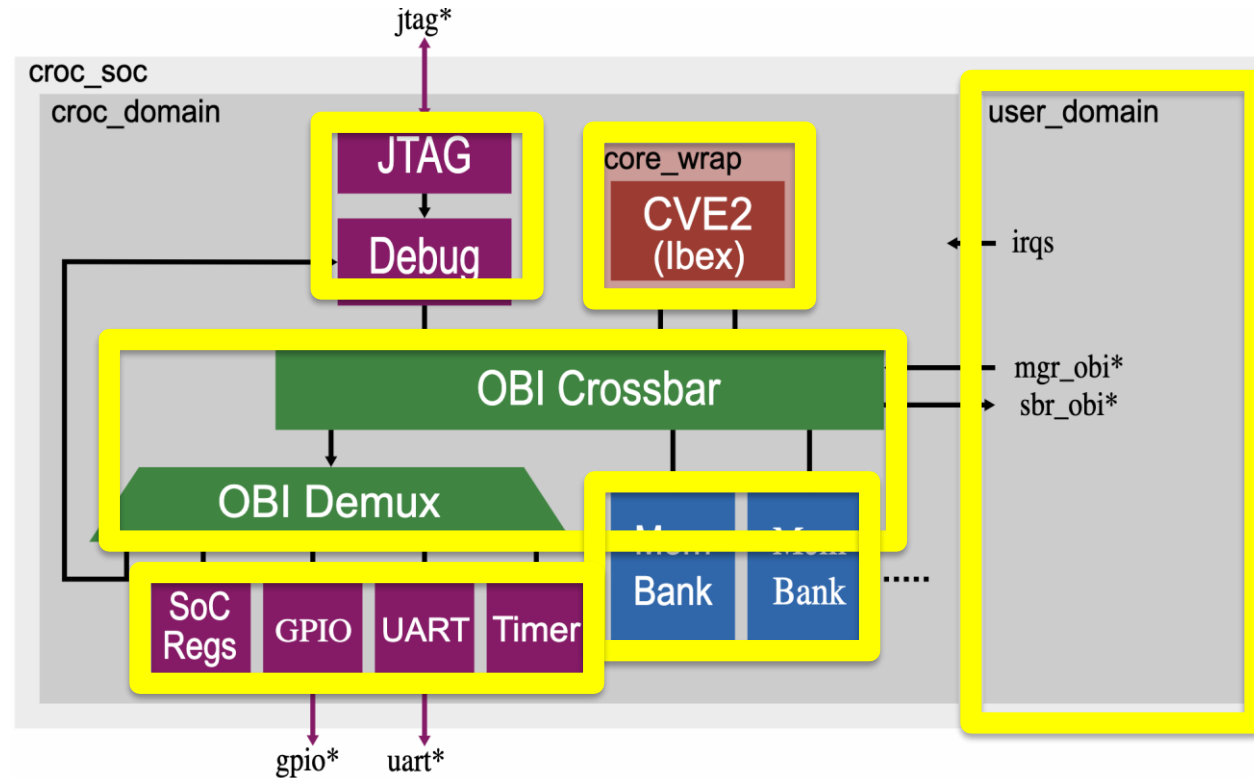
- ❑ Both data and instruction

❑ Debug and programming interface (JTAG)

- ❑ The way to access the SoC and program it

❑ Peripherals

❑ User Domain



Simplifications in Croc that make our life easier

- **Smaller memory can be kept as fast as the core**
 - Croc is not a large chip, so we can afford to have a smaller memory
 - The access speed of the memory and the pipeline are (more or less) balanced
- **The OBI interconnect allows multiple ports and banks**
 - Both Data and Instruction memory are connected to the same interconnect
 - The banks have different addresses
 - As long as there is no address conflict, they can be read/written simultaneously
- **The JTAG interface allows us to read and write to memory locations**
 - We can load the binary we have compiled into the memory where we want
 - Then we can start the execution of Croc